TITLE

UNDERGRADUATE FINAL YEAR PROJECT

Hans Sebastian
2022390007

Computer Science
Faculty of Engineering annd Technology
Sampoerna University
Jakarta
2025

TITLE

UNDERGRADUATE FINAL YEAR PROJECT

Hans Sebastian
2022390007

Supervisor 1: **Supervisor 2:**

Supervisor 1 Name with title **Supervisor 2 Name with title**

2025

CHAPTER I

INTRODUCTION

1.1 Background

1.1.1 Blueprinting and Other Traditional Copying Methods

Before digital technology became widespread, engineers, architects and designers used conventional methods such as mylars and blueprints which require the drawings and measurements to be copied manually by dozens of workers. Blueprints utilized chemicals such as potassium ferricyaniide and ferric ammonium citrate solution to coat the original drawing to be copied. The coated paper is then exposed to ultraviolet light, creating a white-on-blue copy after washing it with water [1]. This method suffers from 2 main drawbacks which limits productivity and flexibility when designing. One such drawback is the modification issue, which is that the master design to be fixed before copying, meaning any changes to the original causes the copies to be deprecated and the entire process redone from scratch. The other issue comes with the need to dry the copies before they become usable, creating more overhead.

Whiteprinting attempted to replace the original blueprinting technique by utilizing ammonia fumes instead of water, resulting in a black-on-white copy that does not need to be dried [2]. Although it improved efficiency by creating a dry copy, whiteprinting includes the creation of hazardous fumes such as ammonia which could cause long-term damage to the operator [3]. Furthermore, this technique still suffer from the modification issue that plagued the original. These downsides pushed engineers to develop better, more efficient alternatives.

1.1.2 Computer Aided Design

Advances in technology have allowed for the development of better procedures, especially computer aided design (CAD) which largely alleviated the issues present in prior methods. CAD can be defined as the use of digital computers to create, modify, analyze, and optimize existing designs [4]. In other words, CAD is the process of utilizing digital technology to perform the entire design process, from ideation to generation and sharing, bypassing the need for traditional copying procedures such as blueprinting. The concept originated from the first program *Sketchpad* designed by Ivan Sutherland in 1963 [5], and has received numerous improvements with newer tools and programs such as SolidWorks [6]. A short diagram detailing the CAD process can be seen in Figure 1.1.

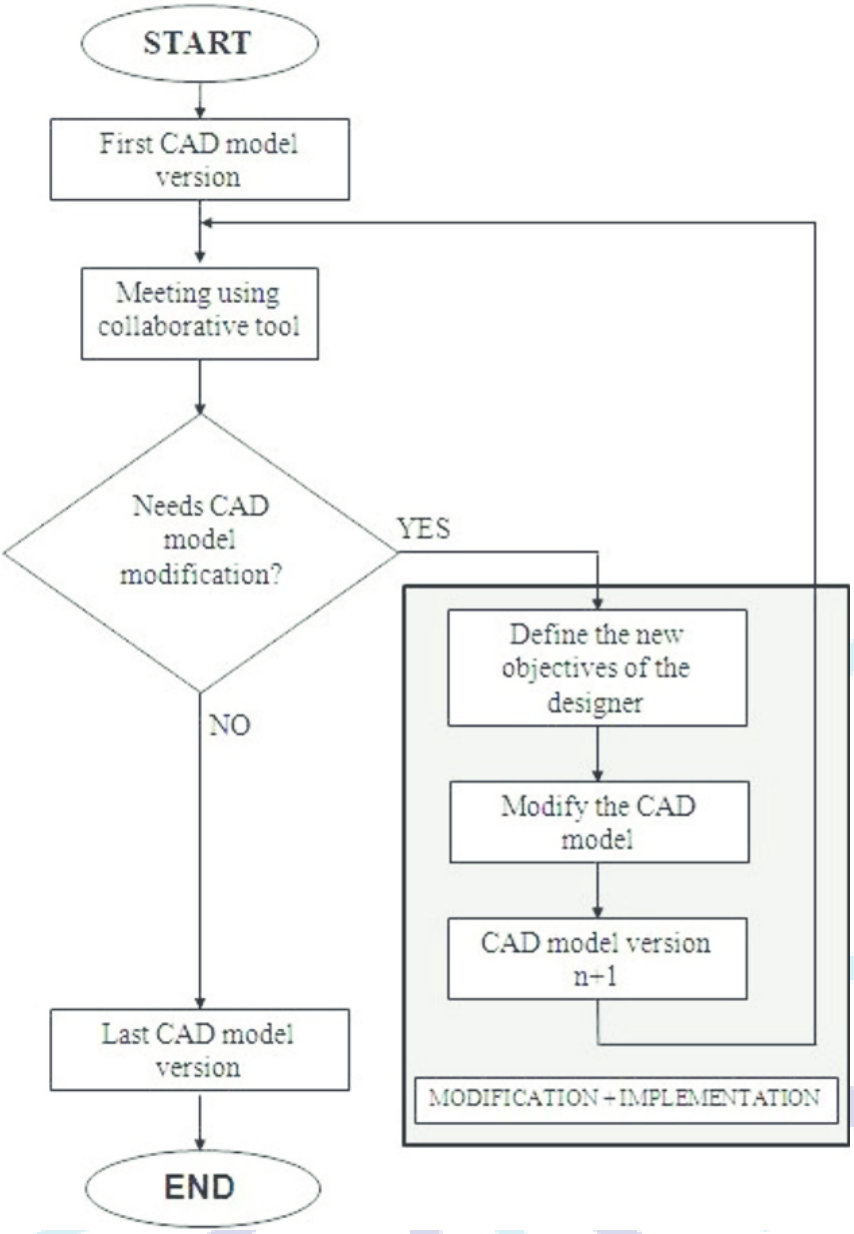

Figure 1.1: Flowchart of the CAD modeling process [7].

In the current era, the use of CAD tools and software have largely replaced such methods as they lack the limitations that come with traditional techniques. This is especially true in engineering-related fields with one study showing up to 50% reduction in task duration [8, 9]. Other benefits include the ability for immediate changes even during late-stage development which helps limit delays [10]. The main benefit of CAD tools is ability to reutilize/repurpose existing CAD designs to be reused in other CAD projects, resulting in more efficient, reliable workflows that eliminates the need to redesign already existing components from scratch [9].

Although the use of CAD have improved the overall design and manufacturing process, they also pose their own set of issues. For example, CAD software

developers often employ their own proprietary file formats, each with their own complex structures. This results in major incompatibility issues as users might have different tooling preferences. As a result, international governing bodies such as International Organization for Standardization (ISO) have developed universal CAD file formats such as STEP to handle the issue of compatibility. A STEP file consists of components that each describe geometric data using a language called EXPRESS [11]. Another universal CAD file format include STL designed by 3D Systems in 1987, which contains a set of triangles consisting of facets and vertices [12]. Another problem comes with difficulties in documentation and indexing for existing CAD designs, as the complexity of the file's structure inhibit the use of automated annotation and indexing tools, forcing the use of manual annotation. Although the universal file formats help with compatibility between different CAD software, issues regarding documentation and indexing still persists, prompting the research for potential solutions.

1.2 Problem Formulation

There are multiple issues relating to CAD design annotation, one example being that manual annotation incurs an unnecessary overhead, diverting manpower and time away from the main design process. Furthermore, manual annotation also introduce risks regarding annotation quality and accuracy, as human annotators might miss important details or mislabel important components, reducing overall workflow efficiency [13]. Difficulties in determining annotation formats and standards increases risk of errors, which is especially noticeable when dealing with third-party designs.

The development of artificial intelligence (AI) have resulted in new techniques for automating the process, however most AI-based methods and models require the use of additional conversion steps into images and/or point clouds to leverage existing image processing and computer vision techniques [13]. This added steps reduce the overall efficiency and increases the risk of misidentification/misclassification. This study aims to develop a novel method for processing CAD files which preserves the existing semantic context and connections between the parts without the need for additional conversion steps into visual representations. Additionally, current state-of-the-art (SotA) models such as Gemini 3 and Cha:t-GPT 5.1 overly rely on the existing metadata contained in the file, completely bypassing the geometry of the design. Other issues related to these models include privacy and latency concerns due to the necessity for an internet connection, as well as high computational and energy costs as the models are designed for general use [14].

1.3 Research Objectives

Based on the topic discussed in the Problem Statement section, this study aims to complete the following research objectives:

1. Generate accurate description for given CAD STEP file using novel transformer-based model.
2. Accurately translate geometric data (points, lines, shapes) into human language.
3. Enhance CAD file indexing by leveraging model-generated descriptions as semantic metadata.
4. Enable low latency inference using compact, topology-aware tokenizer.

1.4 Significance of Study

This study's significance and contributions towards existing literature are detailed as follows:

1. Enable faster development and project organization with automatic STEP CAD file annotation.
2. Develop a topology-first serialization strategy for STEP file B-rep graph
3. Advance existing classification models from general classification to generating detailed descriptions.

1.5 Limitation of Study

The following study lists the following limitations and potential improvements:

1. **Context-dependence:** The labeling of CAD objects relies on designer intent, which might be lost during pre-processing.
2. **Implicit Bias:** The correctness and quality of the output and training process are determined based on human responses, which introduces potential bias.
3. **Limited Vocabulary:** The model's output and training data are based solely on the English language, limiting its ability to generate annotations in different languages.
   Additionally, the model's vocabulary is modified to reduce subjectivity by limiting the use of adjectives such as "big" or "small".
4. **Sparse classes:** The model's training data only consists of general common objects which can be easily found in public datasets in order to maximize the accuracy of the model. This is due to the scarcity of complex CAD designs in such datasets as they contain mostly reusable assets.

1.6 Organization of the report

The rest of the paper will be organized according to the following structure: Chapter II analyses the existing literature and basic terminologies required to understand the topic. Chapter III details the methodology used to gather the training dataset and the design considerations related to model development,

from tokenizing, fine-tuning, to the benchmarking process. Chapter IV lists the results gathered during the benchmarking process paired with detailed analysis of said results. Lastly, Chapter V summarizes the conclusion obtained from the study as well as potential improvements for future research.

CHAPTER II

LITERATURE REVIEW

2.1 Data and Information

2.1.1 Definitions

Data refers to a collection of raw, unlabelled information which represents a set of truth/facts about the world [15]. Data comes in two main categories, quantitative and qualitative [16]. Quantitative data refers to data which has a distinct form, with unique categories and can be analyzed using objective numerical operations and methods, where qualitative data lacks a concrete structure and is highly context-dependent and subjective [17]. Quantitative data mostly consists of values that can be represented numerically (a person's height/weight, price of an object), while qualitative data refers to data about a subject such as nominal descriptions (a person's name, a novel's synopsis). This paper focuses on digital data, which is data represented as a sequence of symbols, mainly binary digits.

The concept of data differs from information in that data serves as the foundation from which information is generated [16]. In essence, information is a specific interpretation of a given dataset, derived from their type and values, as well as their context and semantic meaning. In particular, the context in which a collection of data is acquired determines the set of valid interpretations of said data, which in turn affects the possible inferences i.e. potential information gathered from it [18]. Raw data in itself cannot be used directly, as it merely represents the attributes of a certain subject, without assigning to it any valid implication/insight. To make use of data, it needs to be converted into information through analysis and contextualization [19].

As the name suggests, information is the basis of all modern information technologies and systems such as search engines, recommendation systems, and artificial intelligence. These systems utilize information gathered from large datasets to gain knowledge, derive solutions and make decisions such as giving a custom-tailored recommendation to particular set of users in the case of recommendation systems, and generating responses in the case of AI. The amount of data generated daily has grown exponentially in the information age, with one estimate stating approximately 402.74 million terabytes are generated by electronic devices around the world daily, and an estimated 394 zettabytes will be generated by 2028 [20]. This prompts the need to develop a method for organizing, analyzing, and managing large batches of data efficiently and accurately.

2.1.2 Data classification

Data classification is the process of assigning labels to data for better management purposes [21, 22]. In short, data classification is the process of grouping data into different categories according to their characteristics for later processing and bookkeeping. Data classification allows for enhanced protection, security and risk management of data based on sensitivity and regulatory standards [23, 24, 21]. Furthermore, data classification allows for better organization and retrieval of certain types of data, improving the accuracy and performance of information gathering. Data classification methods largely follows this specific sequence of steps: 1) defining classification criteria, 2) data discovery and sensitive data detection and 3) data labeling based on type, usage, and security requirements [22].

Most forms of data, such as text, images, and video/audio recordings, are categorized as qualitative as they lack a formal, fixed set of attributes and structure. Furthermore, data with complex structures such as CAD files contain a large amount of features and attributes that form deep semantic relationships. As a result, these types of data cannot be easily classified using conventional methods as they rely on assumptions regarding the data's structure. Recent advancements in artificial intelligence systems such as natural language processing and deep learning have allowed for reliable, efficient classification of these datatypes [25]. In modern systems, vector embeddings have become the cornerstone that enables these capabilities, replacing the less efficient rule-based approach in natural language processing, and acting as the basis for deep learning systems such as convolutional/recurrent neural network models [26, 27]. This paper will focus on the use of vector embeddings as a method for enumerating and processing CAD files for classification purposes.

2.2 Vector Embeddings

2.2.1 Definition and Purpose

Vector embedding is defined as a method for converting and representing complex, unstructured data as a sequence of numerical values [28, 29]. A vector embedding aims to create a compact, structured representation of an underlying datapoint while preserving the underlying context and semantic meanings contained in it [29]. In other words, vector embeddings encode unstructured, context-rich data into points in an n-dimensional space, capturing the relationships of different data points in numeric form, allowing the data to be processed and analyzed using mathemathical operations [28]. This representation also renders the concept of "distance" as a quantifiable component in data analysis, where distance refers to semantic proximity/closeness in meaning [30]. In short, the distance between n-dimensional vector representation of two datapoints closely models the semantic relationship between the original data.

Vector embedding serves as a way to efficiently encode large datapoints such as texts, images, video and audio into a digestible, compact format that

could be easily processed by other systems. Vector embeddings also allows data to be searched, indexed, and grouped using their semantic similarity [31]. One use case is in large language models, where models could be equipped with the ability to access external documents and data, allowing it to produce reliable output grounded on falsifiable truth [32].

Other similar methods for encoding qualitative, complex data have been developed, such as one-hot encoding, which encodes each datapoint as states represented as a sequence of n-bits, where all but one bit is set to '0', and the '1' bit represents the current state of the system [33]. Another example is Bag of Words (BoW), which represents text as a "bag"; an unordered, unstructured collection of its words, or keypoints in the case of an image [34], and Term Frequency-Inverse Document Frequency (TF-IDF), which encodes words/keypoints as its number of occurrence and rarity inside a document [35].

Nonetheless, the methods listed previously face several limitations when compared to newer techniques such as vector embedding, such as the loss of semantic information such as the word order, grammatical structure, and other relevant context stored in the original. Other problems include the problem of dimensionality, which states that as the number of dimensions increase for a given set of data, the amount of available data becomes increasingly sparse and patterns more obscure [36]. This limitation also massively increases the compute power and memory required to store and process these encodings while also increasing the risk of errors [37]. Vector embedding sidesteps these limitations by mapping similar items to vectors which have a close distance to each other, preserving semantic data and minimizing the number of features required. The shift from sparse, count-based encoding methods towards a more compact, learned encoding technique enables critical performance improvements in terms of data processing metrics such as compute time and output accuracy.

2.2.2 Embedding Models

Although vector embedding offers several important advantages, such advantages require specialized, novel machine learning models that are capable of distilling complex relationships between datapoints into a compact dense vector representation while minimizing the amount of features/dimensions. These models capture context ingrained within datapoints by learning how to assign a connection between two neighboring words/parts of data [38, Chapter 4.1].

Most embedding models could be categorized into two groups: static models that generate a single, fixed representation for a word/data segment, such as continuous BoW and Subword models [39], and contextual models such as Bidirectional Encoder Representations from Transformers (BERT) and its variants that generate unique representations of the same word depending on the current context [40, 41]. In recent times, contextual models have largely dominated due to their flexibility and better capabilities in terms of capturing the nuance and implicit interconnections between datapoints and its components.

One popular example of this is the transformer architecture, which forms the basis the paper's research.

2.3 Transformer Architecture

2.3.1 Architecture Overview

Current state-of-the-art (SotA) embedding models are based on the transformer architecture, which acts as the foundation for commercial generative AI models such as ChatGPT, Gemini, DeepSeek, etc. Introduced in the seminal paper "Attention Is All You Need" by Vaswani [42], instead of processing data sequentially, the architecture utilized a novel approach for sequence-processing tasks referred in the paper as self-attention, replacing the recurrent, in-order structures of previous models like recurrent neural networks (RNNs) and long short-term memory models (LSTMs) [42, 43, 44].

The models share their similarity in their use of tokenization, a process of dividing input data into smaller chunks referred to as tokens. The tokens are represented as numeric vectors, reducing the amount of storage and compute needed for further processing. The literature dictates two main paradigms of tokenizing input data, full word and subword tokenization. Full word tokenization, as it implies, splits input data based on a pre-defined separator, such as whitespace in text. The above method have largely been replaced by its counterpart, which divides input data into smaller subparts that has no fixed separator. Examples of subword tokenization include byte pair encoding and WordPiece [45, 46, 41]. The tokens generated from the process are then used as the main input data of the self-attention mechanism.

The self-attention mechanism detailed in the transformer architecture improves upon previous systems by enabling models to evaluate the importance of the surrounding parts of an input sequence, regardless of its ordering. For instance, consider the sentence "The dog barked at the cat to drive it away." Using self-attention, transformer-based models can correctly associate the word "it" with the word "cat" rather than "dog" [42]. This capability of handling remote dependencies and understand context allows it to outperform previous architectures. The details of the mechanism lies in the novel components that make up the architecture, forming a deep network. In particular, the transformer model's functionality and reliability is the outcome from combining two key components, encoders and decoders.

2.3.2 Encoder-Decoder

The encoder-decoder components consists of several blocks/subcomponents that in turn form the basis of a transformer layer, which are then stacked continuously to build the complex models. The encoder functions as a processing layer for the entire input sequence, aiming to build a rich, contextualized numerical representation utilizing the self-attention mechanism. The encoder is implemented as two main parts:

1. **Multi-Head Self-Attention:**
   The self-attention of the transformer is employed as different units/heads working in parallel, allowing the model to learn different types of relationships from the input data simultaneously [47]. Each "head" is tasked with learning a different aspect of the given sequence. For instance, one head might focus on syntactic relationships while another learns semantic ones. This functionality is done through projecting the input data into different representation subspaces and applying self-attention in parallel. The final result is a vector generated by applying a linear transform to the concatenation of the heads' output [42]
2. **Position-wise Feed-Forward Networks (FFN):**
   The next step consists of passing the token vectors gathered and weighted from the attention mechanism through a feed-forward neural network. The subcomponent consists of two linear neural network layer with a non-linear activation function in between, such as ReLU. It enriches each token's representation by first expanding the vector's dimension (i.e. 512 to 2048), applying an activation layer and resizing the vectors back into its original size. This process allows the model to extract deep, interconnected patterns hidden within the data. Furthermore, the network acts as a key-value store for patterns learned during training, and its output represents a distribution of tokens containing said patterns [48].

The parallel nature of self-attention makes it difficult to retain information regarding sequence ordering. Hence, positional encodings in the form of special vectors are added to the embedddings to mitigate this limitation. The positional encoding vectors are generated using sinusoidal functions with different frequencies that represent positions of tokens inside the sequence [42]. After the encoding process, the result generated by the encoder is passed through a decoder that remaps the resulting vectors back into tokens that forms the final output sequence. The decoder achieves this through three steps:

1. **Masked Multi-Head Self-Attention:**
   The first step of the decoding sequence is similar with encoding, with a key difference being he lack of a lookahead for future tokens. This is done to prevent model overfitting as knowing future tokens might embed irrelevant connections into the model.
2. **Cross-attention:**
   Another change made in the decoding process is the addition of a cross-attention layer. The layer acts as a critical link between the encoder and decoder that functions as a window for the decoder to look at the encoder's final output. The encoder uses its internal state as a query in the form of a vector and relates it to the vector set generated by the encoder. This process enabels the decoder to determine the most relevant parts of the input and its relation to the final output [42].
3. **Position-wise Feed-Forward Networks (FFN):**
   The final decoding step is similar to the encoding process with no changes.

Enabling effective training of densely-layered networks require proper integration of the encoder and decoder. To meet these requirements, two opera-

tions are applied post-encoding and decoding to ensure stability and effective training. The integration involves adding a residual connection layer that adds the original input vector with the output to minimize the vanishing gradient problem. Next, the vectors are normalized and stabilized, ensuring a smoother and reliable training process [42]. The architectural decisions that make up the model allows the connections between each token to be filtered and strengthened based on the input's context, and allows the model to retain more information from its training data [42].

2.4 Related Works

Early attempts at classifying CAD files mainly used feature descriptors such as Zernike and Reeb graph descriptors to determine the category of a CAD model based on shape and function [49].

Several other works focuses on 2D based methods such as utilizing machine learning to classify over 1600 CAD files using random forest, support vector and fully-connected neural network models by identifying 45 different basic shapes [50], with subsequent approaches combining GNN to measure and group CAD files based on their similarities in geometry [51]. More examples of this type include RotationNet, which utilizes images of the same 3D objects from different angles to estimate its pose and category [52].

Different category of related studies uses a 3D-based approach, with one leveraging the use of 3D grid representation (voxels) of the CAD file and applying a 3-dimensional-CNN that could detect and categorize features [53], and another applies a Mask-recurrent CNN model to extract 3D objects from a single 2D RGB image which can then be classified [54]. Works such as PointNet utilizes 3D point clouds to estimate the class of an object [55]. A different classification uses a custom 3D representation named PANORAMA and feeding the result to a CNN to find keypoints and classify the CAD based on the detected features [56, 57, 58]. Newer 3D-based approaches utilizes graph-based techniques to make use of the graph-like structure of CAD files such as STEP, with one work using Graph-based Neural Network (GNN) to categorize certain CAD STEP files into 7 different classes [13]. However, a research gap in previous works can be identified, since the cited works are limited to classifying a fixed class of CAD objects, such as bolts, and cannot be used to create a general description of objects not specified in the class dataset.

Another method applies a Feature Directed Acyclic Graph (FDAG) to determine the dependency between the shapes of CAD models which are then optimized until only the fundamental features and shapes remain. The shapes can then be compared to CAD models to determine their resemblance to the shapes used as a query [59]. One graph based method also utilizes the structure of STEP files to generate an attribute graph which can then be used as a measure for shape similarity [60, 61]. Although the methods above are able to determine other objects with similar shapes, the method relies on the user's knowledge

of the general shapes that define each object category. This paper aims to fill identified research gaps mentioned through implementing a novel framework for analyzing CAD files using transformer based architecture.

CHAPTER III

PROPOSED METHODOLOGY

3.1 Data Gathering

3.1.1 Dataset Source Selection

Publicly available datasets for CAD file designs can be found on the internet. One prominent example is the ABC dataset, which consists of over one million unannotated STEP files derived from Onshape [**abc_dataset_koch**]. Another is the Fusion 360 Gallery, a dataset of free user-submitted designs with design timeline [**fusion_dataset_willis**], and Text2CAD, which hosts around 170,000 CAD models with 660,000 natural language annotations ranging from short descriptions to detailed explanations [**text2cad_mohammad**]. Other sources include free 3D/CAD design websites such as GrabCAD and TraceParts, which contain various CAD objects not found in compiled datasets. A comparative summary for the sources listed above is contained in Table 3.1 below.

Based on the analysis detailed in Table 3.1, this paper will utilize a combination of CAD designs contained in Text2CAD as well as other online sources such as the ABC dataset and TraceParts. This combination ensures a balance between labeled semantic data (Text2CAD) and high-fidelity geometric variety (TraceParts). To help minimize discrepancies between annotations of several different datasets, a standard for annotation format mimicking the existing annotations in the Text2CAD dataset will be used. A summary of the procedure can be seen in **??**

3.1.2 Dataset Preprocessing

3.1.2.1 Dataset Filtering

Before training, the dataset must be analyzed and filtered to remove low-quality CAD designs that could negatively affect the model's output. The analysis of the dataset is managed using Python with the `step-utils` and **Pandas** modules. A critical preprocessing step is **Manifold Validation**. As STEP files may contain disjointed surfaces or unstitched faces, any file failing a "watertight" topological check (i.e., containing open shells) is discarded to prevent such outliers from polluting the model. Additional steps such as ensuring a balanced dataset (similar number of instances per class) to ensure statistical robustness and prevent model overfitting. More specifically, the class distribution is analyzed to identify and prune 'long-tail' categories, i.e. classes containing

Source	Type	Summary	Advantages	Disadvantages
TraceParts	Online Website	A library of standard engineering parts (bolts, gears, bearings) verified by manufacturers.	High-fidelity geometry, standard-compliant naming conventions.	Limited to standard parts; requires scraping.
GrabCAD	Online Website	Community repository with 6+ million user-generated CAD files.	High diversity of complex assemblies and creative designs.	Inconsistent quality; requires filtering for non-manifold geometry.
Text2CAD	Compiled Dataset	A collection of CAD files of different engineering parts with standardized annotation.	Ready-to-use text-geometry pairs; strictly organized.	Limited geometric complexity compared to industry files.
Fusion 360	Compiled Dataset	Gallery of designs exported from Autodesk Fusion 360, focusing on assembly structures.	Contains reconstruction history (timelines).	Proprietary format issues; inconsistent export quality.
ABC	Compiled Dataset	Large-scale collection of 1M+ unlabelled B-Rep files from Onshape.	Massive scale useful for pre-training; robust topology.	Lack of semantic labels; contains many 'garbage' or tutorial files.

Table 3.1: Comparison between various dataset sources.

fewer than 100 unique geometric instances. Furthermore, deep learning models often resort to 'memorization' rather than generalization to handle classes with insufficient sample sizes [**deep_learning_goodfellow**].

Research also shows that machine learning models struggle with identifying classes with low occurrences/high variability where the class pattern breaks [**memorization_effect_you**]. The final dataset distribution is detailed in Section 3.2.

In Deep Learning, classes with insufficient sample sizes often lead to 'memorization' rather than generalization; the model learns to identify the specific training examples via noise patterns rather than learning the underlying topological features that define the class. By enforcing a minimum threshold of $N = 100$, we ensure that every semantic category (e.g., `GEAR`, `BRACKET`) possesses enough geometric variance to allow the Transformer to learn an abstract, generalized representation of the object type

3.1.2.2 Annotation Format

Since raw STEP files from sources like ABC do not contain semantic labels, a semi-automated annotation pipeline is employed. For TraceParts data, part names (e.g., "Hex Head Screw ISO 4017") are parsed and mapped to a simplified taxonomy of classes (e.g., `FASTENER`, `GEAR`, `BRACKET`). For Text2CAD, existing JSON-based annotations are retained. The final target label Y for any given input X is a categorical token representing the functional class of the 3D object.

3.2 Model Design

This research proposes a novel *Hierarchical Dual-Stream Tokenizer* designed specifically to address the non-Euclidean nature of B-Rep (Boundary Representation) data. Unlike standard Natural Language Processing (NLP) tokenizers that treat input as a 1D string, the proposed method treats the STEP file as a topological graph that must be normalized, compressed, and linearized before processing.

3.2.1 Tokenizer Architecture

The tokenization pipeline consists of four distinct stages designed to ensure geometric invariance and computational efficiency.

3.2.1.1 Stage 1: Semantic Macro-Compression

Raw STEP files suffer from extreme verbosity, where simple features (e.g., a hole) are described by dozens of low-level surfaces. To mitigate this, a graph contraction algorithm is applied. A rule-based parser scans the B-Rep graph for known topological sub-structures. For instance, a `CYLINDRICAL_SURFACE` bounded by two `CIRCLE` edges is collapsed into a single macro-node `<CYLINDER_PRIMITIVE>`. This reduces the sequence length by approximately 80% while retaining semantic meaning.

3.2.1.2 Stage 2: Canonical Geometric Normalization

Standard STEP coordinates are sensitive to global positioning; a bolt at $x = 0$ has different coordinate tokens than a bolt at $x = 100$. To achieve translational and rotational invariance, the B-Rep graph undergoes **Canonical Alignment**:

1. **Centering:** The centroid of the solid is calculated, and all control points P_i are translated such that the centroid lies at $(0,0,0)$.
2. **Alignment:** Principal Component Analysis (PCA) is performed on the vertex cloud. The entire geometry is rotated via a rotation matrix R such that the first principal component aligns with the global Z-axis.

3.2.1.3 Stage 3: Topological Linearization (Canonical DFS)

Since Transformers utilize positional encoding to interpret sequence order, the serialization of the B-Rep graph must be deterministic. A standard Depth-

First Search (DFS) is insufficient because it is sensitive to the arbitrary indexing of nodes in the STEP file (e.g., visiting Face #10 before Face #20 purely based on ID).

To resolve this, we implement a **Canonical DFS**. This algorithm introduces a strict ordering constraint at every branch of the traversal. For any parent node N_p with a set of child nodes $C = \{c_1, c_2, ..., c_k\}$, the children are sorted based on a geometric invariant function $f(c)$ before traversal:

$$\text{Order}(C) = \text{sort}_{\downarrow}(\{f(c) \mid c \in C\}) \tag{3.1}$$

In this research, the invariant function $f(c)$ is defined as the **Surface Area** for topological faces and **Curve Length** for edges. This ensures that a specific 3D geometry yields an identical token sequence regardless of the permutation of entity IDs in the raw file, effectively removing graph isomorphism ambiguity from the input data.

3.2.1.4 Stage 4: Complex-Valued Dual-Stream Encoding

To preserve high-fidelity precision (e.g., NURBS weights) without bloating the vocabulary, a Dual-Stream architecture is used. The tokenizer emits two parallel streams for every step t:

- **Stream A (Semantic Tokens):** Discrete integers representing the type of entity (e.g., `<FACE>`, `<NURBS_CURVE>`).
- **Stream B (Geometric Embeddings):** A 128-dimensional **Complex-Valued Vector** encoding the shape parameters.

The complex vector $v \in \mathbb{C}^{64}$ encodes the geometry as $z = r + i\theta$, where the real part r represents scale-invariant magnitudes (e.g., radius, edge length) and the imaginary part θ encodes relative orientation using Rotary Positional Embeddings (RoPE).

3.2.2 Model Parameters

The core model is a Transformer Encoder based on the BERT architecture, modified to accept the Dual-Stream input.

- **Embedding Dimension (d_{model}):** 512. The discrete tokens are embedded into $\mathbb{R}^{512}$, and the 128-dim complex geometric vectors are projected via a Multi-Layer Perceptron (MLP) to $\mathbb{R}^{512}$ before summation.
- **Attention Heads:** 8 Parallel heads.
- **Layers:** 12 Encoder blocks.
- **Context Window:** 2048 tokens (sufficient due to the Macro-Compression stage).
- **Feed-Forward Network:** GELU activation with a dimension of 2048.

The selection of these hyperparameters is governed by the specific trade-offs inherent to B-Rep graph learning:

While LLMs often utilize dimensions upwards of 4096, CAD syntax has a significantly smaller vocabulary size (< 1000 unique structural tokens) compared to natural language. A dimension of $d_{model} = 512$ was chosen to prevent overfitting on the sparse vocabulary while providing sufficient manifold capacity to represent the fused Dual-Stream input. Specifically, it ensures that the projection of the 128-dimensional complex geometric vector does not dominate the semantic signal of the discrete token embedding during the summation process.

The choice of 12 layers aligns with the hierarchical nature of boundary representation. Lower layers (L_{1-4}) are tasked with learning local topological syntax (e.g., edge continuity), intermediate layers (L_{5-8}) aggregate these into feature-level representations (e.g., pockets, bosses), and upper layers (L_{9-12}) extract high-level semantic class information. The 8 parallel attention heads allow the model to disentangle the multi-modal input: specific heads can learn to attend solely to the "Real" component of the input (spatial magnitude) while others attend to the "Imaginary" component (orientation) and topological connectivity, mimicking a multi-view perception system.

Standard BERT implementations utilize a 512-token window. However, B-Rep graphs—even after the proposed Macro-Compression—exhibit long-range dependencies where a face defined at token T_1 may reference a surface defined at token T_{1000}. A window of 2048 tokens was empirically selected to encompass the complete topological graph of 95% of the dataset samples without necessitating destructive truncation, striking a balance between memory constraints and geometric completeness.

3.2.2.1 Geometry-Infused Attention Mechanism

Unlike standard NLP Transformers where the attention score is derived solely from semantic word embeddings, the proposed architecture employs a **Geometry-Infused Attention** mechanism. This mechanism is responsible for synthesizing the Dual-Stream output from the tokenizer into a unified representation.

The input to the attention layer for the i-th token is defined as the fusion of the semantic and geometric streams:

$$X_i = \text{Embed}(T_i) + \text{MLP}(V_i) \tag{3.2}$$

Where T_i is the discrete semantic token (e.g., `<FACE>`) and V_i is the 128-dimensional complex geometric vector. The Multi-Layer Perceptron (MLP) projects the continuous geometric manifold into the model dimension d_{model}.

3.2.2.1.1 Rotary Positional Integration To strictly enforce the rotational invariance defined in the methodology, the model utilizes a modified version of Rotary Positional Embeddings (RoPE). The "Phase" component (θ) of the complex geometric input V_i is directly mapped to the rotation of the Query (Q) and Key (K) vectors in the attention calculation:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{(R_{\theta_i}Q)(R_{\theta_j}K)^T}{\sqrt{d_k}}\right)V \tag{3.3}$$

Here, R_θ represents the rotation matrix derived from the tokenizer's complex input. This ensures that the attention score between two CAD entities depends not only on their semantic type but also on their **relative geometric orientation**. For example, a `SCREW` entity will attend strongly to a `HOLE` entity only if their orientation vectors (Normal vectors) are aligned (i.e., the relative rotation is near zero), thereby embedding physical assembly logic directly into the attention weights.

[Image of multi-head attention architecture diagram]

The model output is passed through a global average pooling layer followed by a softmax classifier corresponding to the number of classes defined in the dataset preprocessing stage.

3.3 Training and Backtesting

3.3.1 Training Setup

The classification objective is optimized using **Categorical Cross-Entropy Loss**. This function penalizes the divergence between the predicted probability distribution $\hat{y}$ and the ground truth one-hot vector y. For a dataset of N samples and C classes, the loss $\mathscr{L}$ is defined as:

$$\mathscr{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c}) \tag{3.4}$$

Where $\hat{y}_{i,c}$ is the softmax probability output of the model for class c. The logarithmic nature of the loss function imposes a heavy penalty on confident but incorrect predictions, which is critical for distinguishing between visually similar CAD classes (e.g., distinguishing an ISO-4014 Bolt from an ISO-4017 Bolt based on subtle shaft length variations).

3.3.2 Evaluation Metrics

To validate the effectiveness of the proposed tokenizer, the model is evaluated on two fronts:

1. **Standard Classification Metrics:** Accuracy, Precision, Recall, and F1-Score on a held-out test set (20% of the data).
2. **Rotational Robustness Test:** A specialized test set is generated by applying random random $SO(3)$ rotations to the test data. The model's accuracy on this rotated set is compared to the baseline to verify the efficacy of the Canonical Alignment and Complex-Valued Embeddings.

BIBLIOGRAPHY

[1] J. Herschel, *On the Action of the Rays of the Solar Spectrum on Vegetable Colours, and on Some New Photographic Processes* (On the Action of the Rays of the Solar Spectrum on Vegetable Colours and on Some New Photographic Processes). Taylor, 1842. [Online]. Available: https://books.google.co.id/books?id=hsBHV4j9QWwC.

[2] F. Parmeggiani, "Fear of the dark: Diazo printing by photochemical decomposition of aryldiazonium tetrafluoroborates," *Journal of Chemical Education*, vol. 91, pp. 692–695, Feb. 2014. DOI: 10.1021/ed400555a. Accessed: Jul. 24, 2025.

[3] J. D. McGlothlin, "Health hazard evaluation report: Hhe-80-248-791, national oceanic and atmospheric administration, u.s. dept of commerce, washington, d.c.," *Health Hazard Evaluation Reports (HHE)*, pp. 1–10, Jan. 1981. DOI: 10.26616/nioshhhe80248791. Accessed: Nov. 28, 2025. [Online]. Available: https://www.cdc.gov/niosh/hhe/reports/pdfs/80-248-791.pdf.

[4] I. Sadrehaghighi, "Computer aided design (cad)," Jan. 2022. DOI: 10.13140/RG.2.2.12634.62408.

[5] I. E. Sutherland, "Sketch pad a man-machine graphical communication system," in *Proceedings of the SHARE design automation workshop*, 1964, pp. 6–329.

[6] I. Zeid, *Mastering Solidworks*. Macromedia Press, 2021.

[7] D. Fleche, J.-B. Bluntzer, M. Mahdjoub, and J.-C. Sagot, "First step toward a quantitative approach to evaluate collaborative tools," in Dec. 2014, vol. 21, pp. 288–293. DOI: 10.1016/j.procir.2014.03.160.

[8] M. Rahman, M. Khatib, P. Rani, S. Shaikh, and G. Gunashekar, "Effect of computer aided drafting on manual drafting skills," *International Journal of Innovative Technology and Exploring Engineering*, vol. 8, pp. 3012–3014, Sep. 2019. DOI: 10.35940/ijitee.K2315.0981119.

[9] S. Gutiérrez de Ravé, E. Gutiérrez de Ravé, and F. J. Jiménez-Hornero, "Enhancing efficiency and creativity in mechanical drafting: A comparative study of general-purpose cad versus specialized toolsets," *Applied System Innovation*, vol. 8, pp. 74–74, May 2025. DOI: 10.3390/asi8030074. Accessed: Jun. 12, 2025. [Online]. Available: https://www.mdpi.com/2571-5577/8/3/74.

[10] A. Aranburu, D. Justel, and I. Angulo, "Reusability and flexibility in parametric surface-based models: A review of modelling strategies," *Computer-Aided Design and Applications*, vol. 18, pp. 864–874, Apr. 2021. DOI: 10.14733/cadaps.2021.864-874.

[11] M. J. Pratt, "Introduction to iso 10303—the step standard for product data exchange," *Journal of Computing and Information Science in Engineering*, vol. 1, pp. 102–103, Jan. 2001. DOI: 10.1115/1.1354995. Accessed: May 4, 2022.

[12] M. Szilvsi-Nagy and G. Mátyási, "Analysis of stl files," *Mathematical and Computer Modelling*, vol. 38, Oct. 2003. DOI: 10.1016/S0895-7177(03)90079-3.

[13] L. Mandelli and S. Berretti, *Cad 3d model classification by graph neural networks: A new approach based on step format*, arXiv.org, Oct. 2022. Accessed: Oct. 13, 2025. [Online]. Available: https://arxiv.org/abs/2210.16815.

[14] J. O'Donnell and C. Crownhart, *We did the math on ai's energy footprint. here's the story you haven't heard.* MIT Technology Review, May 2025. Accessed: Nov. 28, 2025. [Online]. Available: https://www.technologyreview.com/2025/05/20/1116327/ai-energy-usage-climate-footprint-big-tech/.

[15] K. Olson, "What are data?" *Qualitative Health Research*, vol. 31, no. 8, 1015960, 2021. DOI: 10.1177/10497323211015960.

[16] L. Hackman, P. Mack, and H. Ménard, "Behind every good research there are data. what are they and their importance to forensic science," *Forensic Science International: Synergy*, pp. 100 456–100 456, Feb. 2024. DOI: 10.1016/j.fsisyn.2024.100456.

[17] J. Schoonenboom, "The fundamental difference between qualitative and quantitative data in mixed methods research," *Forum Qualitative Sozialforschung / Forum: Qualitative Social Research*, vol. 24, Jan. 2023. DOI: http://dx.doi.org/10.17169/fqs-24.1.3986. Accessed: Oct. 13, 2025. [Online]. Available: https://www.qualitative-research.net/index.php/fqs/article/view/3986/4916.

[18] L. Mason, "Data vs information," *www.academia.edu*, Jan. 2019. [Online]. Available: https://www.academia.edu/39505535/Data_vs_Information.

[19] S. Borrohou, R. Fissoune, and H. Badir, "The role of data transformation in modern analytics: A comprehensive survey," *Journal of Computer Languages*, vol. 84, p. 101 329, 2025, ISSN: 2590-1184. DOI: https://doi.org/10.1016/j.cola.2025.101329. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2590118425000152.

[20] P. Taylor, *Data growth worldwide 2010-2028*, Statista, Jun. 2025. Accessed: Oct. 12, 2025. [Online]. Available: https://www.statista.com/statistics/871513/worldwide-data-created/?srsltid=AfmBOoos3sZoZKw6aS7wf2CiXFwWvkUGV6VUzUBHutfpJE82JTtKg53o.

[21] W. Newhouse, M. Souppaya, J. Kent, K. Sandlin, and K. Scarfone, "Data classification concepts and considerations for improving data protection," *Data Classification Concepts and Considerations for Improving Data Protection*, Nov. 2023. DOI: 10.6028/nist.ir.8496.ipd. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/ir/2023/NIST.IR.8496.ipd.pdf.

[22] S. K. Shivayogi, "Data classification methodologies and implementation," *Journal of Computer Science and Technology Studies*, vol. 7, pp. 202–210, May 2025. DOI: 10.32996/jcsts.2025.7.5.26. Accessed: May 31, 2025.

[23] A. Lane and J. B. Adelusi, *Role of data classification in enhancing security in big data environments*, ResearchGate, Jul. 2023. [Online]. Available: https://www.researchgate.net/publication/388067206_Role_of_Data_Classification_in_Enhancing_Security_in_Big_Data_Environments.

[24] H.-N. Nguyen and L. Cuong, "Overview of data classification and applications in data security," *International Journal of Environmental Sciences*, vol. 11, pp. 31–39, Jun. 2025. DOI: 10.64252/ac0b3685. Accessed: Jul. 20, 2025.

[25] M. Blessing, "Ai-driven data classification and organization," Sep. 2021.

[26] R. Patil, S. Boit, V. Gudivada, and J. Nandigam, "A survey of text representation and embedding techniques in nlp," *IEEE Access*, vol. 11, pp. 36 120–36 146, 2023. DOI: 10.1109/access.2023.3266377.

[27] D. S. Asudani, N. K. Nagwani, and P. Singh, "Impact of word embedding models on text analytics in deep learning environment: A review," *Artificial Intelligence Review*, vol. 56, pp. 1–81, Feb. 2023. DOI: 10.1007/s10462-023-10419-1.

[28] P. Pajo, "Vector embeddings unveiled: A comprehensive exploration of their creation, types, applications,...," *ResearchGate*, Apr. 2025. DOI: 10.13140/RG.2.2.15544.05129. [Online]. Available: https://www.researchgate.net/publication/390598346_Vector_Embeddings_Unveiled_A_Comprehensive_Exploration_of_Their_Creation_Types_Applications_Challenges_and_Future_Directions_in_Machine_Learning/.

[29] K. Chitturi, "Understanding vector embeddings in ai search: A technical deep dive," *INTERNATIONAL JOURNAL OF COMPUTER ENGINEERING & TECHNOLOGY*, vol. 15, pp. 1725–1733, Dec. 2024. DOI: 10.34218/IJCET_15_06_147.

[30] S. Kukreja, T. Kumar, V. Bharate, A. Purohit, A. Dasgupta, and D. Guha, "Vector databases and vector embeddings-review," pp. 231–236, 2023. DOI: 10.1109/IWAIIP58158.2023.10462847.

[31] T. Taipalus, "Vector database management systems: Fundamental concepts, use-cases, and current challenges," *Cognitive Systems Research*, vol. 85, p. 101 216, 2024, ISSN: 1389-0417. DOI: https://doi.org/10.1016/j.cogsys.2024.101216. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389041724000093.

[32] O. Omolayo and O. Babatope, "Comparative evaluation of vector embedding frameworks for scalable sematic retrieval in pdf-based rag systems," *International Journal of Research Publication and Reviews*, vol. 6, pp. 12 506–12 511, Jun. 2025. DOI: 10.55248/gengpi.6.0625.23100.

[33] A. I. J. I. B. W. Samuels, "One-hot encoding and two-hot encoding: An introduction," Jan. 2024. DOI: 10.13140/RG.2.2.21459.76327.

[34] W. Qader, M. M. Ameen, and B. Ahmed, "An overview of bag of words;importance, implementation, applications, and challenges," pp. 200–204, Jun. 2019. DOI: 10.1109/IEC47844.2019.8950616.

[35] S. Qaiser and R. Ali, "Text mining: Use of tf-idf to examine the relevance of words to documents," *International Journal of Computer Applications*, vol. 181, Jul. 2018. DOI: 10.5120/ijca2018917395.

[36] D. L. Banks and S. E. Fienberg, *Data Mining, Statistics*, Third Edition, R. A. Meyers, Ed. New York: Academic Press, 2003, pp. 247–261, ISBN: 978-0-12-227410-7. DOI: https://doi.org/10.1016/B0-12-227410-5/00164-2. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B0122274105001642.

[37] G. V. Trunk, "A problem of dimensionality: A simple example," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-1, pp. 306–307, Jul. 1979. DOI: 10.1109/TPAMI.1979.4766926. Accessed: Dec. 29, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/4766926.

[38] H. Tran, *Natural Language Processing for Everyone*. Bookdown, 2024. [Online]. Available: https://bookdown.org/tranhungydhcm/mybook/.

[39] B. Wang, A. Wang, F. Chen, Y. Wang, and C.-C. Kuo, "Evaluating word embedding models: Methods and experimental results," *APSIPA Transactions on Signal and Information Processing*, vol. 8, Jan. 2019. DOI: 10.1017/ATSIP.2019.12.

[40] Y. Liu, G. Tilahun, X. Gao, Q. Wen, and M. Gervers, *A comparative study of static and contextual embeddings for analyzing semantic changes in medieval latin charters*, 2025. Accessed: Oct. 12, 2025. [Online]. Available: https://aclanthology.org/2025.loreslm-1.14.pdf.

[41] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds., Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423. [Online]. Available: https://aclanthology.org/N19-1423/.

[42] A. Vaswani et al., "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17, Long Beach, California, USA: Curran Associates Inc., 2017, pp. 6000–6010, ISBN: 9781510860964.

[43] A. Sherstinsky, "Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network," *Physica D: Nonlinear Phenomena*, vol. 404, p. 132 306, Mar. 2020. DOI: 10.1016/j.physd.2019.132306.

[44] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, pp. 1735–1780, Nov. 1997. DOI: 10.1162/neco.1997.9.8.1735. [Online]. Available: https://direct.mit.edu/neco/article-abstract/9/8/1735/6109/Long-Short-Term-Memory?redirectedFrom=fulltext.

[45] B. Suyunu, E. Taylan, and A. Özgür, *Linguistic laws meet protein sequences: A comparative analysis of subword tokenization methods*, 2024. arXiv: 2411.17669 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2411.17669.

[46] L. Kozma and J. Voderholzer, *Theoretical analysis of byte-pair encoding*, 2024. arXiv: 2411.08671 [cs.DS]. [Online]. Available: https://arxiv.org/abs/2411.08671.

[47] E. Voita, D. Talbot, F. Moiseev, R. Sennrich, and I. Titov, *Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned*, 2019. Accessed: Oct. 12, 2025. [Online]. Available: https://aclanthology.org/P19-1580.pdf.

[48] M. Geva, R. Schuster, J. Berant, and O. Levy, "Transformer feed-forward layers are key-value memories," *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds., pp. 5484–5495, Jan. 2021. DOI: 10.18653/v1/2021.emnlp-main.446. Accessed: Oct. 10, 2023. [Online]. Available: https://aclanthology.org/2021.emnlp-main.446/.

[49] C. Y. Ip and W. C. Regli, "Automatic classification of cad models," Apr. 2005. DOI: 10.17918/etd-505. Accessed: Oct. 25, 2025.

[50] D. Machalica and M. Matyjewski, "Cad models clustering with machine learning," *Archive of Mechanical Engineering*, vol. 66, no. 2, pp. 133–152, Apr. 2019. DOI: 10.24425/ame.2019.128441. Accessed: Oct. 25, 2025. [Online]. Available: https://yadda.icm.edu.pl/baztech/element/bwmeta1.element.baztech-77666cdd-ada0-472e-b608-86fafaa40164.

[51] R. Roj, M. Sommer, H.-B. Woyand, R. Theiß, and P. Dueltgen, "Classification of cad-models based on graph structures and machine learning," *Computer-Aided Design and Applications*, vol. 19, pp. 449–469, Sep. 2021. DOI: 10.14733/cadaps.2022.449-469.

[52] A. Kanezaki, Y. Matsushita, and Y. Nishida, "Rotationnet: Joint object categorization and pose estimation using multiviews from unsupervised viewpoints," *arXiv:1603.06208 [cs]*, Mar. 2018. Accessed: Nov. 13, 2021. [Online]. Available: https://arxiv.org/abs/1603.06208.

[53] D. Maturana and S. Scherer, "Voxnet: A 3d convolutional neural network for real-time object recognition," in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015, pp. 922–928. DOI: 10.1109/IROS.2015.7353481.

[54] C. Gümeli, A. Dai, and M. Nießner, *Roca: Robust cad model retrieval and alignment from a single image*, arXiv.org, Jun. 2022. DOI: 10.1109/CVPR52688.2022.00399. Accessed: May 13, 2024. [Online]. Available: https://arxiv.org/abs/2112.01988.

[55] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, *Pointnet: Deep learning on point sets for 3d classification and segmentation*, arXiv.org, 2016. [Online]. Available: https://arxiv.org/abs/1612.00593.

[56] P. Papadakis, I. Pratikakis, T. Theoharis, and S. Perantonis, "Panorama: A 3d shape descriptor based on panoramic views for unsupervised 3d object retrieval," *International Journal of Computer Vision*, vol. 89, pp. 177–192, Aug. 2009. DOI: 10.1007/s11263-009-0281-6. Accessed: Dec. 7, 2022.

[57] K. Sfikas, T. Theoharis, and I. Pratikakis, "Exploiting the PANORAMA Representation for Convolutional Neural Network Classification and Retrieval," in *Eurographics Workshop on 3D Object Retrieval*, I. Pratikakis, F. Dupont, and M. Ovsjanikov, Eds., The Eurographics Association, 2017, ISBN: 978-3-03868-030-7. DOI: 10.2312/3dor.20171045.

[58] K. Sfikas, I. Pratikakis, and T. Theoharis, "Ensemble of panorama-based convolutional neural networks for 3d model classification and retrieval," *Computers & Graphics*, vol. 71, pp. 208–218, 2018, ISSN: 0097-8493. DOI: https://doi.org/10.1016/j.cag.2017.12.001. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0097849317301978.

[59] M. Li, Y. Zhang, J. Fuh, and Z. Qiu, "Design reusability assessment for effective cad model retrieval and reuse," *International Journal of Computer Applications in Technology, 2011*, vol. 40, pp. 3–12, Feb. 2011. DOI: 10.1504/IJCAT.2011.038546. Accessed: Oct. 13, 2025. [Online]. Available: https://www.inderscience.com/info/inarticle.php?artid=38546.

[60] M. El-Mehalawi and R. Allen Miller, "A database system of mechanical components based on geometric and topological similarity. part i: Representation," *Computer-Aided Design*, vol. 35, pp. 83–94, Jan. 2003. DOI: 10.1016/s0010-4485(01)00177-4. Accessed: Apr. 25, 2022.

[61] M. El-Mehalawi and R. Allen Miller, "A database system of mechanical components based on geometric and topological similarity. part ii: Indexing, retrieval, matching, and similarity assessment," *Computer-Aided Design*, vol. 35, pp. 95–105, Jan. 2003. DOI: 10.1016/s0010-4485(01)00178-6. Accessed: Apr. 25, 2022.